Endoforge

Software creation as a runtime operation

Edition 3 · Oct 17, 2026 · @Craig

1. The idea in one page

Endoforge makes the creation of software an operation a running system performs, under rules that let the result be trusted without trusting the thing that wrote it.

A person or an agent brings a need. The runtime finds existing capabilities, imports useful code, or generates what is missing; checks, runs, and measures it in a bounded workcell; qualifies it against an evaluator the candidate never saw; and supplies a working capability. Ordinary use then executes code instead of spending another model turn.

The thesis, in the order that matters:

1. The expensive thing is repeated understanding, not repeated compilation. Work should be proportional to the change and to the newly required evidence, never to the size of the project.
2. Proposals are cheap now. The surrounding system exists to turn cheap proposals into trusted capabilities at the lowest total cost, and total cost includes the model, the repairs, the evaluation, the review, and the failures.
3. A RAM-resident workcell inside one isolate is the enabler that keeps the repeated loop local. It is a means to the first two points, not the idea itself.

Four commitments define the design and separate this edition from its predecessors:

- **Specification is a subsystem.** Every task gets an evaluator synthesized before any candidate exists, composed mostly from reusable invariants, qualified against fixed meta-gates, and confirmed in plain language by a human for its novel part.
- **Trust is a measured quantity.** What earns trust is the harness, never the model: the number of independent error sources a promotion crosses, and the measured rate at which seeded faults escape it.
- **The candidate is a pure kernel plus declarative glue.** Computation ships as a WebAssembly module with declared imports; views, bindings, and tests ship as data validated by schema. No generated imperative JavaScript executes anywhere.

- **The model writes all of the code**, including the trusted base, and none of it from a position where it grades its own work.

The unit of success is an independently accepted useful capability per unit of total cost. Tokens generated, isolates created, and checks per second explain a result; they are not the objective.

What should feel different: a person asks for a capability, confirms in a sentence what will be checked, and is using working software within a minute or two. The current version stays pinned while a better one is qualified. A restart restores the capability from durable records. A failed experiment never replaces a working version.

This is a design document. Recorded results are marked as such; everything else is proposed. Section 14 is the entire evidence base. Section 15 is the shortest path from here to the first number that matters.

2. How to read this document

Every number and every architectural statement in this document carries one of five labels, and the label matters more than the number.

Label	Meaning	Example in this document
Recorded	Reported by a receipt from a real run, not rerun for this edition	Warm guest median 18.64 ms, Field OS main rig, 8 Oct 2026
Upstream	Reported by a third-party maintainer, pinned where possible	tsc-rs native 2.45× TypeScript 7 on T3 Code, 13 Oct 2026
Derived	Arithmetic from stated values	669 / 24.04 = 27.8× cold-to-warm ratio
Estimated	An order-of-magnitude figure reasoned from public data, not measured here	A 100-line kernel is about 1,000 tokens, 5–10 s to generate
Proposed	Intended architecture, experiment, or rule; nothing establishes it yet	The whole of Sections 4 to 13 unless marked otherwise

Recorded does not mean rerun. Upstream native compiler timings are not WebAssembly timings. A proposed architecture is not a deployed capability. None of those should be read as the one beside it.

What changed since Edition 2 (16 October 2026):

- The thesis is reordered. Change-proportional work leads; in-isolate locality is an enabler, not the headline.
- The two products are separated. The agent substrate and generative applications share an architecture but not an evidence bar, and the plausibility of the first no longer carries the second.
- Specification has a theory. Evaluator synthesis is a first-class stage with its own rules and its own qualification.
- Trust has a definition and a number. The harness is the unit of trust, and the escape rate on seeded faults is how much of it exists.
- The candidate changed shape. A pure WebAssembly kernel plus declarative glue replaces generated JavaScript in a nested interpreter, and the QuickJS layer leaves the architecture.
- The model may author the trusted base, under boundaries of authorship time, context, and privilege rather than a human-versus-model rule.
- Dependency risks are named: the age and provenance of tsc-rs, the 128 MB Workers isolate ceiling, shared-process side channels, and retained capabilities as injection vectors.
- Physical-limits material is reduced to its two engineering consequences and moved out of the main line.

The Edition 2 engineering library, the original white paper, and the thirty-sheet atlas remain the record of the earlier design. Where this edition disagrees with them, this edition governs.

3. Two products, one substrate

Endoforge is two products with opposite economics sharing one architecture, and the design is only honest when it says which one each claim is about.

	Agent substrate	Generative applications
What it is	A cheap, trusted iteration loop for coding agents and tool-building agents	An application that grows new capabilities on request, inside a stable interface
Customer	Agent developers and platform teams	End users of a host application, and the host's developers

	Agent substrate	Generative applications
Where the evaluator comes from	The agent's own task: existing tests, a fixed benchmark, a build that must pass	Synthesized per request (Section 6); no evaluator exists in advance
Trust required	Modest: a wrong candidate fails the task visibly	Maximal: a wrong capability is promoted, retained, and reused
Dominant cost	Model inference per iteration	Specification and review per request
Competitors	Warm-sandbox services, agent harnesses, snapshot-restore microVMs	Configurable software, spreadsheets, notebooks, conventional feature development
Horizon	Near term; incremental advantage, measurable in weeks	Long term; depends on the specification cost falling
Evidence bar	Accepted candidates per second per unit cost against a warm native baseline	Real users completing real tasks with manufactured capabilities, at lower total cost than the alternative

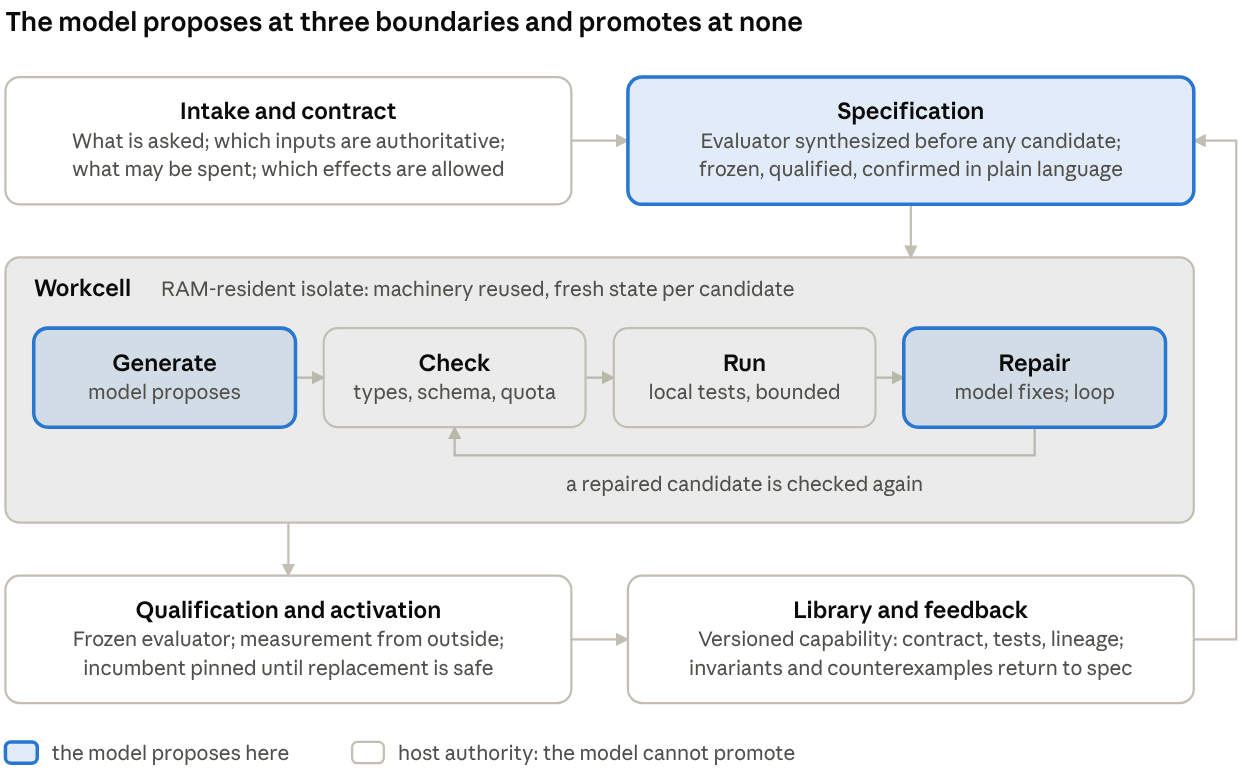
The first product is an infrastructure bet. Its differentiators are density, locality, and a harness that lets an agent run many bounded experiments without an environment per attempt. It can be measured against existing sandboxes within weeks (Section 15).

The second is a product bet whose critical dependency is not the substrate at all. It is whether a request from a user at the "too small for a project" scale can be turned into a checkable contract cheaply enough. Sections 6 and 7 are the answer this edition gives; Section 15 names the experiment that tests it.

Field OS remains the applied setting and the only source of recorded evidence. It exercises the agent substrate today and is the intended first host for generative applications. Its refrigeration domain, its Python authority plane, and its native stack are not the definition of either product.

4. The production cycle

One request crosses five boundaries, and each boundary changes who is allowed to do what.



production cycle · 5 boundaries, 1 repair loop, 1 feedback path

Specification precedes the workcell, qualification sits outside it, and the library feeds the next request's evaluator rather than the next request's code.

Intake and contract. The host establishes what is being asked, which inputs are authoritative, what resources may be spent, and which effects are permitted. Imported code is material, not authority. If the request is underdetermined, the host asks; no amount of synthesis supplies a missing fact.

Specification. Before any candidate exists, an evaluator is synthesized for the task from reusable invariants and the request itself, qualified, frozen, and confirmed (Section 6). This stage is new in this edition and it precedes the workcell on purpose.

The workcell. A RAM-resident isolate holds prepared machinery: a pinned compiler for each supported route, approved dependencies, a private overlay per candidate, and the cheap deterministic gates. Candidates are generated, checked, run against local fixtures, and repaired here. Machinery is reused; every candidate's mutable state is fresh.

Qualification and activation. Survivors leave the workcell as exact artifacts and meet the frozen evaluator, resource measurement from outside the isolate, and the promotion rule. Activation is a guarded revision change: the incumbent stays pinned until the replacement can take over without losing state, focus, or authority.

Library and feedback. An accepted capability becomes a versioned entry with its contract, tests, applicability, and lineage. Its invariants and counterexamples return to the specification stage, which is how later requests get cheaper. Its code is the least valuable thing it leaves behind (Section 12).

The model participates at three of the five boundaries: it frames the contract, it synthesizes the evaluator, and it generates and repairs candidates. It participates at none of them from a position that can promote its own output.

5. The candidate is a pure kernel plus declarative glue

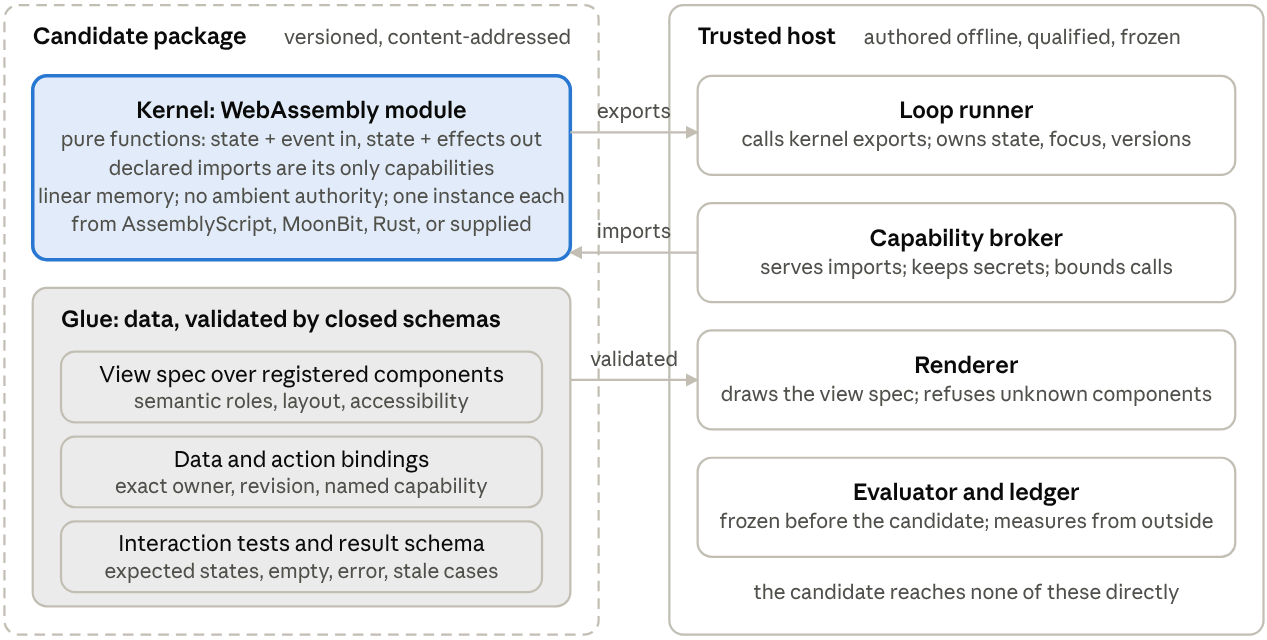
Every candidate is a pure WebAssembly kernel plus declarative glue, and no generated imperative JavaScript executes anywhere in the system.

The kernel is a WebAssembly module exporting pure functions: state in, event in, state out, effects out as data. Its declared imports are its only capabilities. It gets linear-memory isolation and no ambient authority, instantiates in well under a millisecond, and costs kilobytes to low megabytes per candidate (estimated) where a nested QuickJS guest cost tens of megabytes (recorded, Section 14).

The glue is data: a view specification over the host's registered components, data and action bindings that name exact owners and revisions, interaction tests, and a result schema. Closed schemas validate it. The host renders it and runs the event loop. The candidate never touches the renderer, the broker, or the ledger.

Three things follow. The inner candidate boundary of the earlier design, a JavaScript interpreter nested inside the factory isolate, becomes the module boundary itself, and the three-level nesting collapses to two. Pure reducers are the easiest software there is to test deterministically and to mutation-test, which feeds Section 6 directly. And the density that Edition 2 could only propose becomes arithmetic: a resident candidate is a module instance, not a heap.

Computation is a module with declared imports; everything else is data



The module boundary is the inner trust boundary: there is no interpreter between the kernel and the host.

candidate anatomy · kernel, three glue artifacts, four host services

The kernel exports functions and imports capabilities; the glue is validated and rendered; nothing the candidate contains can call the broker, the renderer, or the ledger except through the host.

Route into the kernel contract	Compiler location	Output	First-pass yield (estimated)	Role
AssemblyScript	JavaScript-hosted compiler, inside the isolate	Small WASM module	Below TypeScript until measured; models emit constructs it lacks	Candidate default for kernels, pending Experiment 1
MoonBit	Browser-hosted compiler, inside the isolate	Small GC-based WASM module	Unknown; little training data	Second in-isolate route; compared in Experiment 1

Route into the kernel contract	Compiler location	Output	First-pass yield (estimated)	Role
Rust	rustc and LLVM, native lane outside the isolate	Small WASM module, strongest static checks, near-native speed	High for pure functions; the borrow checker adds about half a repair round	Keeper refinement; kernels that earn it
TypeScript to JavaScript	tsc-rs as WASM, inside the isolate; execution in a child V8 isolate	JavaScript module	Highest of the four	Compatibility lane for imported code and behavior that resists the kernel split
Pre-built WASM	None; validation only	Module as supplied	Not applicable	Imported components after admission and qualification

Values cross the module boundary through a typed interface. WIT and the Component Model are the intended contract; they are mature in Wasmtime and polyfilled rather than native in V8 hosts, which is one reason the local lane is Wasmtime-based (Section 13). Large inputs stay in host-owned buffers reached through handles; a candidate never receives a pointer into trusted state.

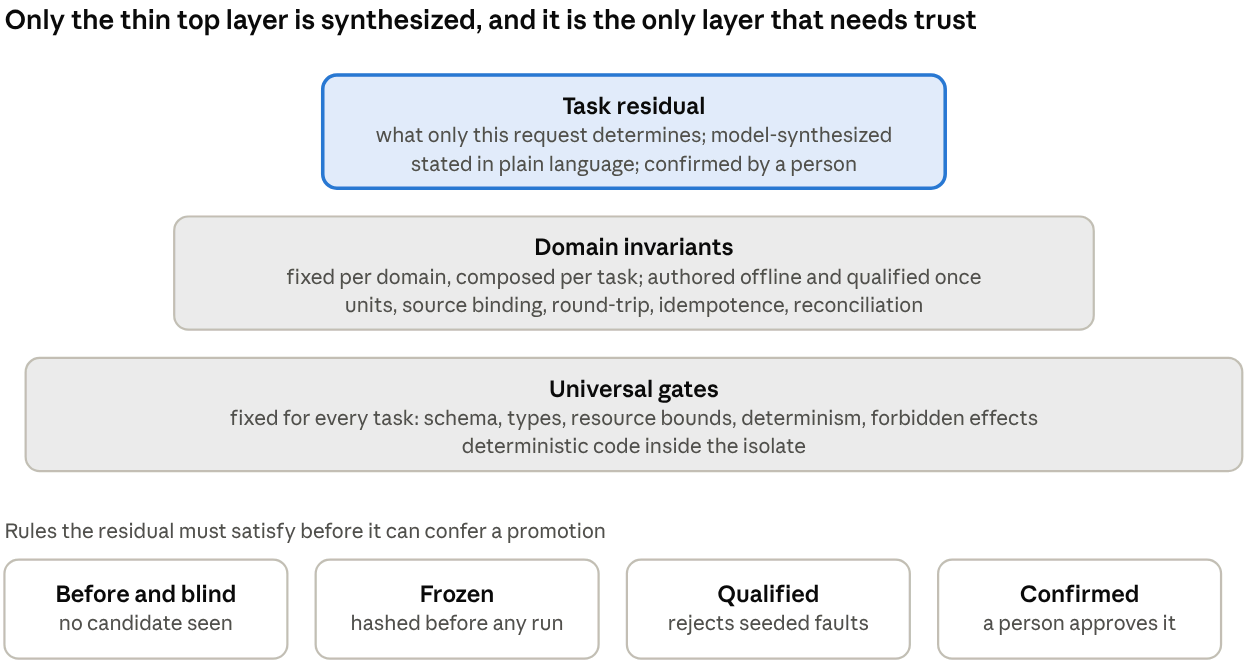
Writing WebAssembly text directly is not a route. Models have almost no training signal for it, a 100-line function becomes a thousand lines of text, and the result cannot be reviewed. Rust-first generation is not the default either: the toolchain cannot live inside the isolate, and per-attempt cost is roughly a wash against TypeScript with a better artifact at the end. That is a refinement profile, not an admission profile.

This section does not require a universal TypeScript-to-WASM compiler, and it does not claim every imported application fits the kernel contract. The compatibility lane exists for exactly those cases, and it is named as a lane rather than disguised as the strict path.

6. Specification is a subsystem

For a novel request there is no fixed evaluator, so the evaluator is synthesized per task; what makes a synthesized evaluator trustworthy is not who wrote it but four rules it must satisfy and the thinness of the part that is actually new.

Acceptance has three layers, and only the top one is synthesized.



acceptance layers · 2 fixed, 1 synthesized, 4 rules on the synthesized one

The lower two layers were qualified before the request arrived; the top layer is the only part that can be wrong in a new way, and the four rules are what keep it from grading itself.

1. **Universal gates**, fixed for every task: contract and schema validity, type and interface conformance, resource bounds, determinism, forbidden effects. Deterministic code, run inside the isolate.
2. **Domain invariants**, fixed per domain and composed per task: dimensional consistency, source-bound return, no cross-room mutation, round-trip and idempotence properties, reconciliation of totals. Authored offline and qualified once. Field OS already holds several. Every accepted capability adds to this layer, which is why Section 12 calls contracts and tests the compounding asset.

3. **The task residual**, the part of acceptance that only this request determines. The model synthesizes it from the request, the data, and any examples the person gives; it is stated in plain language and expressed as tests the host can run.

The design objective is to make the residual thinner every quarter. The thinner it is, the less the synthesis step has to be trusted, and the more of a promotion rests on checks that were qualified before the request arrived.

The residual is a candidate like any other, so it is governed by rules rather than by confidence:

- **Before and blind.** It is synthesized in a context that has seen no candidate, and it is shared by every candidate in the campaign. Unique per task, never per candidate, or nothing can be ranked.
- **Frozen.** Its hash enters the ledger before evaluation begins. It may be revised only on evidence independent of any candidate's outcome; "the test was wrong, let me fix it" after a failure is the most common collapse in agent loops and it is forbidden here.
- **Qualified.** It must reject seeded faults (swapped units, dropped rows, off-by-one, reversed inputs: the shape of the existing six-case fixture), accept a known-good anchor where one exists, and run deterministically within bounds. An evaluator that accepts mutants has no discriminating power and confers nothing.
- **Confirmed.** The person sees "here is what will be checked" before the campaign starts and approves it. Approving a sentence takes thirty seconds; reviewing code takes an hour. For the agent substrate the confirming party is the agent's own task contract.

The rules exist because of correlated error. The failure is not a candidate editing its test; it is the same model reading the request the same wrong way and producing a test and a candidate that agree with each other. First principles make a claim calibrated and honest. They do not manufacture ground truth, and a synthesized evaluator is computation, which cannot supply a fact the request never contained. So the residual's authority is drawn from sources other than the generator's reading of intent wherever possible: the person's own examples, the data itself, differential tests against an existing implementation, contracts inherited from composed capabilities, and metamorphic relations that hold without knowing the right answer.

7. Trust is a measured quantity

The unit of trust is the harness, never the model, and the amount of trust a harness has is the measured rate at which seeded faults escape it.

In security engineering, "trusted" names a component whose failure violates the system's guarantees, which is why a trusted base is minimized rather than expanded. Granting trust to the model's run-time output does not change its error rate; it only removes whatever would have caught the error. Trust is therefore not something the design grants. It is a budget the design measures and spends.

A harness is trustworthy to the degree its checks have independent error sources. Ten checks that all derive from one model turn are one check. The audit question for any promotion is how many independent sources it crossed.

Error source	Checks that draw on it	Independent of the model's reading of intent?
Deterministic tooling	Type and interface checks, schema and contract validation, resource and determinism bounds, forbidden-effect scanning	Yes
Offline-qualified invariants	Domain invariants composed into the task's acceptance (Section 6, layer 2)	Yes
Existing implementations	Differential tests against a known-good function or the incumbent	Yes
Seeded faults	Mutation testing of the synthesized evaluator	Yes
The person	Confirmation of the plain-language residual; approval at promotion where the tier requires it	Yes
The model, candidate-blind	Synthesized residual tests, frozen before the first candidate	Partly: a separate context, the same prior
The model, in the loop	Candidates, repairs, self-generated tests, any model-as-judge step	No

The last row contributes discovery and repair feedback. It contributes nothing to promotion. POODO's own capsule checker verifies the structure of a claim, not its truth, and its repository states that a passing checker does not demonstrate sound reasoning; it is counted as discipline and a receipt schema, not as a check.

The number that makes "harnessed" mean something is the escape rate:

```
\text{escape rate} = \frac{\text{seeded faults promoted}}{\text{seeded faults submitted}}
```

Seed known faults into candidates (swapped units, dropped rows, a wrong join rule, a stale source binding), run them through the full harness, and count how many are promoted. The ratio is measurable this week, it drops every time a non-model check is added, and it sets the autonomy tier per domain instead of a forecast about next year's models.

Autonomy tier	Who promotes	Required escape rate (proposed)	Where it applies first
Confirmed	A person approves the residual and the promotion	Measured and reported; no ceiling	New domains; anything with downstream physical consequence
Supervised	A person approves the residual; promotion is automatic	Below 1 in 1,000 on the domain's fault set	Domains with a mature invariant library
Autonomous	No person in the request	Below 1 in 10,000, and acceptance covered entirely by qualified invariants	Agent-substrate tasks with a fixed task contract

Three properties define the trust boundary, and none of them is human versus model: time of authorship (offline and frozen, or during the task it governs), context separation (whether the checker shares the prior of the thing checked), and privilege (whether a component can change what judges it). Section 8 applies them to the model as author of everything.

Two dynamics make the thresholds strict. Errors in the evaluator are silent: a wrong candidate fails visibly, a wrong evaluator promotes a wrong candidate with a receipt that says it passed. And the swarm is an optimizer: under search pressure, rare evaluator mistakes become the best-scoring path, so an escape rate under random sampling overstates safety under optimization. Section 11 returns to this.

8. The model writes all of the code

The model authors the harness, the evaluators, the invariants, and the candidates; what it never does is write any of them from a position where it grades its own work.

Who types the code was never the trust boundary. A human-written harness is not trustworthy because a human wrote it, and the Edition 2 rule that the trusted harness is "not generated" was a proxy for three properties that can be stated directly.

Boundary	Trusted base (harness, evaluator engine, invariant library)	Per-task evaluator residual	Candidates
Time of authorship	Offline; frozen and versioned before any task uses it	Before the first candidate of its campaign; frozen	During the task
Context separation	Reviewed adversarially in a context that did not write it; tested against oracles from outside its own authoring turn	A context that has seen no candidate	Its own context; sees repair feedback only
Privilege	Changed only through the promotion path, with the current version judging the proposed one and a person approving	Cannot be edited after freezing except on outcome-independent evidence	Cannot reach the evaluator, the broker, the ledger, or its own quotas

Under those boundaries the model can write every line. Changes to the trusted base are a handful of approvals a month, not a bottleneck, and they are the machinery loop of Section 11 done properly: a runtime, compiler, or search-policy challenger is still a candidate under an external acceptance contract.

The compiler this project already depends on is the demonstration. tsc-rs is entirely model-written, and it is trusted here for three reasons that have nothing to do with trust being granted: it was validated against 181,711 tests ported from the Go reference compiler and checked on 120 open-source repositories, an oracle that did not originate in the model that wrote it; it is pinned, frozen, and changed through releases; and it has no authority over anything, since it reads files and emits diagnostics. That is the model writing the code done right. The model rewriting the compiler while compiling, under trust, is what the boundaries above prevent.

POODO has two roles here and one non-role. As the discipline of the proposer, it frames the task, separates the stated from the inferred, names the strongest rival framing, and decides when to stop or ask; those rival framings also seed diverse candidates, which is what makes parallel candidates fail independently (Section 10). As a schema, its precommit record (observable, acceptance boundary, oracle and oracle version, artifact identity, environment, failure meaning, stop condition) is the evaluation receipt of this design, and its separation of delivery state from outcome state is enforced in the ledger by deterministic code. It is not an evaluator. It runs at campaign boundaries, never per candidate: its own scope statement excludes routine execution, and a twenty-path enumeration with a mandatory research gate on every iteration would wreck the economics of the loop.

The same boundaries answer the question of trust the way the project needs it answered. The architecture does not require that agents rarely make mistakes. It requires that candidates may be wrong often, that the harness catches them at a measured rate, and that the one component with nothing above it, the synthesized residual, is thin, blind, frozen, qualified, and confirmed. That is a system whose autonomy can be turned up on evidence as models improve, which is the forward-looking design. A system that assumes the trust is already there is the one that cannot tell when it has been wrong.

9. Three lifetimes and the boundaries that hold

State has three lifetimes with three different rules, and every durability claim in this design reduces to keeping them apart.

Lifetime	What lives there	Rule	Loss means
Durable	Task identity, admitted inputs, the frozen evaluator, candidates kept, accepted artifacts, receipts, application state that must survive restart	Persist before any reference points at it; never a live promise, never a guest heap	Loss of truth; not permitted
Warm	Compiled modules for each route, the approved dependency base, the invariant library, read-only input caches partitioned by permission	Reusable and evictable; a cache hit is never authorization	Higher latency on the next attempt
Attempt	One candidate's overlay, its compiler state, its module instance, its buffers	Fresh per candidate; discarded after bounded execution; outputs published only after validation	Nothing; the task continues from durable records

Four boundary rules carry over from Edition 2 unchanged, because the evidence for them is the only recorded evidence this project has.

An isolate is not a failure boundary. Isolates in one process share that process's death, its memory pressure, and its side channels. Partition cohorts by trust, authority, and acceptable shared failure; keep the controller and the ledger outside every cohort so that losing a process loses attempts, never the task.

Outcome unknown is a state, not an error. An attempt authorized before a crash may have executed. Recovery fences future work, verifies the exact owned closure, and records ambiguity rather than inferring success from leftover output. A process name, a PID, or a successful spool is not proof of completion. Field OS recorded five named recovery boundaries behaving this way (Section 14).

Limits are enforced where they work. A timeout on a blocked event loop cannot stop a synchronous loop. Deadlines, memory, output volume, and host-call amplification are enforced by the runtime, an external watchdog, or termination of an owned cohort, and

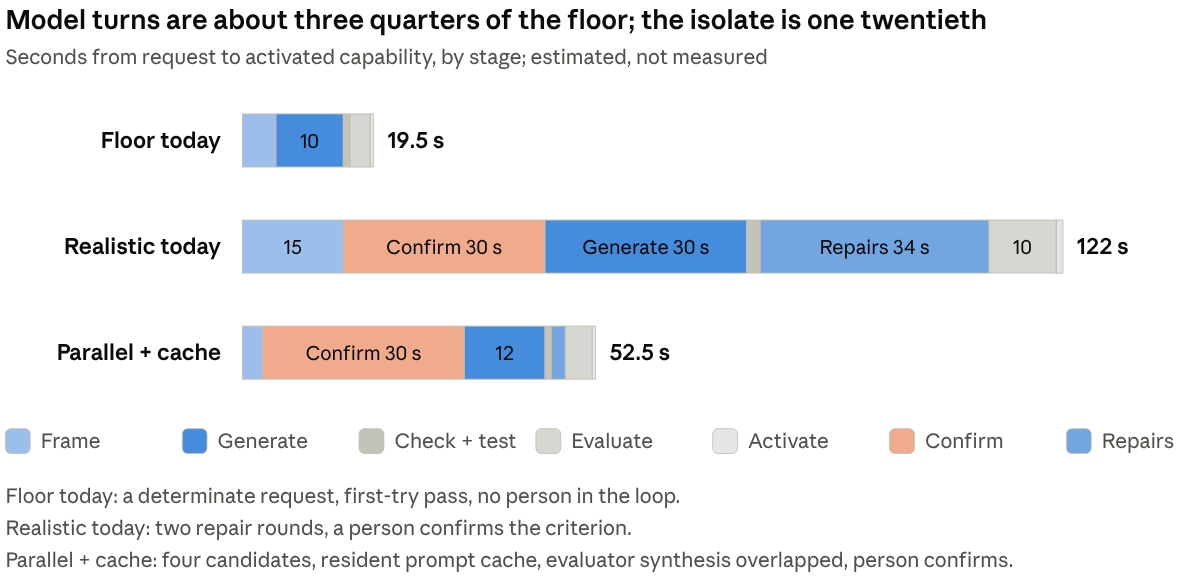
measurement comes from counters the candidate cannot touch.

Waiting releases compute. A task waiting on a person, a model, or a network holds a continuation in durable records and no execution slot. Resumption is a fresh admission: source scope, context, policy, deadline, and runtime are revalidated. A swarm is a queue with a measured number of active slots, not a pile of idle machines.

What changes in this edition is only the attempt row. A module instance replaces a nested interpreter plus its virtual filesystem, so the per-attempt footprint falls by one to two orders of magnitude (estimated; Experiment 4 measures it), and the inner boundary that Edition 2 drew around QuickJS is now drawn by the WebAssembly runtime itself.

10. Where the time goes

For a small novel capability the floor today is about 20 seconds and a realistic run is two to three minutes, and roughly four fifths of either number is model inference rather than anything the workcell does.



estimated stage budgets, this edition · 3 scenarios, 7 stages

The stages the project has engineered, the compiler, the module instance, and the local tests, are one to three seconds in every scenario. The person's confirmation is the only term with an unbounded tail, and it is a property of the request's ambiguity, not of the stack.

The answer therefore has three classes, by request rather than by engineering.

Request class	Today (estimated)	Floor in about two years	What moves it
Composable from the library	0.5–2 s	Unchanged	Retrieval, binding, contract check; no generation at all
Novel and determinate	20 s floor; 2–3 min realistic; 20–25 s with four parallel candidates	3–5 s as output speeds approach 1,000 tokens/s	The model's output speed and first-pass yield; nothing in the factory
Ambiguous	Everything above plus one human round trip: 30 s if present, hours if not	Unchanged	Nothing technical; at the "too small for a project" scale this is most requests

Six levers move these numbers; two of them beat the floor rather than approach it.

- **Generate less.** Structured changes, parameter binding, and composition of existing capabilities in place of fresh source. This matters more for latency than for memory.
- **Start before the request.** Discovery from a failed action or an agent noticing a gap lets the loop run before the person asks; for common patterns, the likely next three capabilities can be manufactured speculatively when a dataset opens. This is the only path to perceived zero.
- **Parallel candidates buy yield, not speed.** At 50% first-pass yield, one candidate expects about one repair round; four give a 94% chance that one passes and cut expected repairs to about 0.07. Sixteen buys 99.998% for four times the cost. Independence requires different framings, which POODO's rival-frame step produces, rather than different seeds on one prompt.
- **Overlap evaluator synthesis with generation.** Blind contexts can run concurrently; the evaluator's hash is frozen before evaluation, not before generation starts. This removes a full model turn from the critical path.
- **Local inference where it pays.** A permanently resident prompt cache turns the SDK context into warm state and cuts time-to-first-token from seconds to tens of milliseconds, three or four times per request. Short calls (retrieval judgments, criterion drafting, single-diagnostic repairs) go local; large generation stays on whichever model has the best yield. Decode is bandwidth-bound: on the order of 100 tokens/s for a 4-bit 7B model on an 800 GB/s part, 30–40 for a 30B, 15–20 for a 70B (estimated), so local is

slower per token for any model large enough to match hosted quality. Batched decode reads the weights once per step for all sequences, so four candidates cost close to one in time; that is the strongest latency argument for unified-memory hardware and it is Experiment 5.

- **Measure the one term never measured.** A model turn against the actual SDK context with prompt caching. Every recorded number in this project is model-free.

The person is on the happy path only when the delivered capability needs no explanation, so legibility (Section 12) is part of the latency, not a separate concern.

11. Search and improvement under a protected contract

The factory may improve the software it makes, the process that finds it, and its own machinery, and it may do none of those by touching the contract that decides whether the improvement is real.

A campaign freezes a task, an evaluator, a budget, and a baseline, then runs a bounded frontier: an incumbent plus diverse parents, a batch of proposals, cheap deterministic rejection first, independent measurement of survivors, and selection that keeps a Pareto frontier rather than one score. Many hypotheses may wait as durable records; only a measured number of candidates hold execution slots. A campaign stops on budget, plateau, regression, or ambiguous execution, and the incumbent is in service throughout.

Loop	What it changes	What it must not change	How a gain is isolated
Product	Algorithms, data structures, layout, state transitions of an accepted capability	The task contract and the invariants	Held-out acceptance plus a matched cost and regression gate against both the incumbent and a fixed reference
Discovery	Proposal strategy, model routing, context, repair feedback, candidate diversity, stopping rules	The evaluator and the budget accounting	Same tasks, same total budget, accepted outcomes compared

Loop	What it changes	What it must not change	How a gain is isolated
Machinery	Compiler bridge, module instantiation, caching, allocation, scheduling	Its own acceptance contract or quotas	Replay of identical inputs through baseline and challenger, then live synthesis under equal budgets

Three facts about optimization shape the rules.

Optimizers find the holes in their objective. That is what optimization is. If synthesized evaluators are wrong one time in a hundred, search pressure does not meet that as a 1% nuisance; it meets the wrong evaluators as the easiest tasks and the candidates that exploit them as the best-scoring. An escape rate measured under random sampling therefore overstates safety under search, and the autonomy tiers of Section 7 are set against the searched rate, not the sampled one.

Parallel samples multiply the evaluator. Sixteen candidates against a 1% leak are not sixteen chances to succeed; they are also sixteen chances to be credentialed wrong. Candidate width is bounded by the evaluator's measured discriminating power before it is bounded by memory.

Repeated feedback trains on the holdout. Scores obtained repeatedly from the same held-out set adapt the search to that set even without access to its contents. Release confirmation uses rotated or fresh fixtures, paired randomized comparisons, repeated measurements, and real host accounting, and an inconclusive difference stays inconclusive.

The arithmetic of where to spend refinement is simple and unforgiving: optimization cost divided by saving per subsequent use, in the same units. Two hundred CPU-seconds spent to save one millisecond per execution need two hundred thousand executions to repay the CPU alone, before model, review, and storage costs. A one-use capsule and a shared compiler path do not get the same budget.

The factory's own search policy can be a challenger. It competes on fixed tasks and total budgets, it cannot alter the evaluator that decides whether it improved, and a change to that evaluator goes through Section 8's promotion path with a person at the end. This is controlled recursive engineering. It is not a claim of unbounded self-improvement, and the document does not make one.

12. The library is inventory

The compounding asset is the growing body of contracts, invariants, tests, and counterexamples; the code is the liability that comes with it.

The factory metaphor's hardest lesson is the one Edition 2 skipped: inventory costs money to hold. Every retained capability carries maintenance against SDK and host churn, re-qualification whenever an evaluator or invariant changes, security review, and a retrieval cost. Retrieval itself, deciding whether a request can be satisfied by composition, is semantic search, which means a model call with the same latency and trust properties as any other. A library with no deprecation, deduplication, or reclamation mechanism is a landfill with provenance.

So the library is structured around what compounds:

- **A capability contract** names what an operation means: schemas, required behavior, declared capabilities, applicability, known failure cases. It is the unit people retrieve and compose.
- **Implementations** are separate artifacts under the contract, each with its own identity, evidence, and resource profile. A low-startup build, a direct-WASM build, and a bounded fallback can coexist. A Rust rewrite is one refinement among several, and the measured improvement is rewarded, not the language label.
- **Qualification records** say which behavior was tested in which environment. They do not transfer: a capability efficient for one input distribution or one machine is not ranked universally.
- **Lineage** follows the data. Generated source can embed confidential values, so a template is split into data-free logic and scoped inputs where possible, and promotion never declassifies what was inlined. Every reuse is a fresh admission under current permissions.
- **Useful failures** are kept as counterexamples and constraints that narrow future search. A rejected candidate that revealed a wrong join rule is worth more than a tenth accepted parser.

Tenant-bound lineage means most user-requested capabilities cannot be shared across tenants, so compounding happens per tenant unless the contracts and invariants, which carry no customer data, are the thing that is shared. That is another reason the contract layer, not the code layer, is the asset.

A plan-phase design should also say how each part of it fares as models improve, because the answer is not uniform.

Component	As models improve	Why
Evaluation plane and harness	More valuable	More untrusted output to check; more autonomy to contain
Substrate (density, locality, bounded execution)	More valuable	More candidates per budget; more experiments per request
Invariant and contract library	More valuable	It is what the synthesized residual shrinks against
Retained code	Less valuable	Regeneration gets cheaper; retrieval plus adaptation plus re-qualification must beat it, and nobody caches a search result
In-isolate locality as a headline	Roughly neutral	Its term in the budget is already small

That table is the reason Section 15 orders the experiments as it does: evaluation plane, then substrate, then library, which is close to the reverse of Edition 2′s emphasis on the workcell.

Legibility belongs here too. Trust in software comes from predictability: a person learns a tool once and it does the same thing for everyone. Manufactured software is new every time, for every person, with no documentation, no community, and no support path when it breaks in six months. The only answer that scales is that a capability is explained by its contract and its evidence, shown to the person in the same plain language they confirmed before the campaign. A capability the person cannot predict is not on anyone′s happy path, however fast it was made.

13. Deployment lanes and named dependency risks

The portable unit is a capability contract plus a checkpoint, and each lane implements it under its own runtime, limits, and enforcement; nothing live, no heap, no credential, no handle, is assumed to move.

Lane	Host	Fits the in-isolate compiler?	Hard limits to design against	Good fit
Local and site	Wasmtime-based runtime under OS sandboxing, cgroups, and an external watchdog; native Rust lane beside it	Yes: AssemblyScript and MoonBit compilers run inside; Component Model native	Memory headroom measured per host; unified-memory bandwidth shared with inference	Private data, offline work, native tools, the first complete workcell
Hosted (Cloudflare Workers)	Factory Worker plus Dynamic Workers for candidates; Durable Objects or an external store for the ledger	Unproven: 128 MB per isolate including WebAssembly memory, 10 MB compressed Worker size on paid plans	Four in-flight Dynamic Workers per request; production timers restricted; no guest self-measurement	Hosted previews and managed dynamic execution, once the compiler's working set is shown to fit
Browser	Module Web Worker with the WebAssembly checker and local workspace; UI thread separate	Yes for small projects	Browser memory and API surface; no renderer-based tests inside a server isolate	Interactive local checks beside the person

The local lane leads, and it is Wasmtime rather than V8 for two reasons: the kernel contract of Section 5 is native there, and the V8 route exists mainly to serve the hosted lane. The hosted lane is kept because it removes a deployment step, not because it has been shown to hold a type-checker, a harness, and a module instance in 128 MB. Experiment 6 decides it.

Six risks are named so they can be tracked rather than discovered.

1. **tsc-rs is days old and machine-written.** Its first public preview shipped on 7 October 2026 as an experimental LLM-written Rust port of the TypeScript 7 compiler, pinned to one upstream revision, with documented stale-output cases under build mode and

memory growth in long sessions. TypeScript has no specification, so parity is a tested property, not a guaranteed one. Microsoft's native compiler is the parity oracle; tsc-rs is pinned by module hash and re-qualified on every bump; and the kernel path of Section 5 does not depend on it at all.

2. **The Workers memory ceiling.** A type-check of a nontrivial project inside 128 MB alongside the harness, the overlay, and a module instance is an open question, and so is single-threaded WebAssembly checking speed against native. Neither number exists yet.
3. **Shared-process side channels.** Isolates in one process are not a Spectre boundary. The hosted platform's mitigations (coarsened timers, dynamic process isolation) do not ship with bare workerd or with any self-hosted runtime. Cohorts of different trust get different processes, and candidate code never receives fine-grained timers.
4. **Retained capabilities as injection vectors.** Data seen at generation time can shape code that later runs on other data, and a promoted keeper is a durable place for that to hide. Review, lineage that follows the data, and re-admission on every reuse are the mitigations; none of them scales for free, which is why review throughput is the binding constraint of the whole design.
5. **SDK and host churn.** Every host contract change invalidates or re-qualifies a slice of the library, and the factory multiplies the number of dependents with no owner. Contracts are versioned; migrations are explicit; a capability with no passing qualification against the current host is retired, not quietly kept.
6. **License and provenance.** The broad intake funnel admits existing scripts, libraries, and model output that may carry license obligations. License is a provenance field from the first admission, not a later audit.

The physical-limits reasoning of Edition 2 survives as two engineering consequences and nothing more: movement can dominate computation (a change-aware path that touches 256 kB instead of 1.28 GB removes a 5,000-fold term, not a 5,000-fold speedup), and when behavior is underdetermined by the available information no computation can supply the missing observation. Everything else in that discussion is background.

14. What is recorded today

The recorded evidence establishes warm guest machinery, bounded execution, custody, and recovery in one private adapter; it establishes nothing about the complete loop, and this section is the whole of it.

Observation	Value	Label and scope
Warm guest, shared VFS, 100% CPU quota	18.64 ms median per job; p95 46.82 ms; peak 150.62 MiB whole unit	Recorded, Field OS main rig, 8 Oct 2026; six-case synthetic fixture; 5 warm-ups + 20 jobs
Warm guest, fresh VFS per job, 100% quota	18.65 ms median; p95 40.49 ms; peak 151.93 MiB	Recorded, same run; the 0.01 ms difference is not statistically meaningful
Preloaded native function, 100% quota	0.131 ms median; peak 29.96 MiB	Recorded, same run; the reviewed function is about 140× cheaper to call natively
Cold lane, 50% quota	Native payload 28 ms vs guest 669 ms; whole supervision 178 ms vs 814 ms	Recorded; cold and warm are distinct observations
Cold-to-warm ratio	669 / 24.04 ≈ 27.8×	Derived; amortizable adapter overhead, not a system speedup
Containment cases	Infinite loop killed at 2,389 ms; allocation bomb at 1,067 ms; output flood timed out at 5,230 ms before proving the transport cap	Recorded, W1/W2 receipt; single fresh-process observations
Recovery boundaries	Five Linux methods across six controller trials in 9.081 s; every ambiguous case recorded as unknown, none as success	Recorded, W3/W4 receipt
Source-bound return	17 named checks passed in 3.425 s; cross-room access refused; stale, revoked, and corrupt outputs withheld	Recorded, W3/W4 receipt

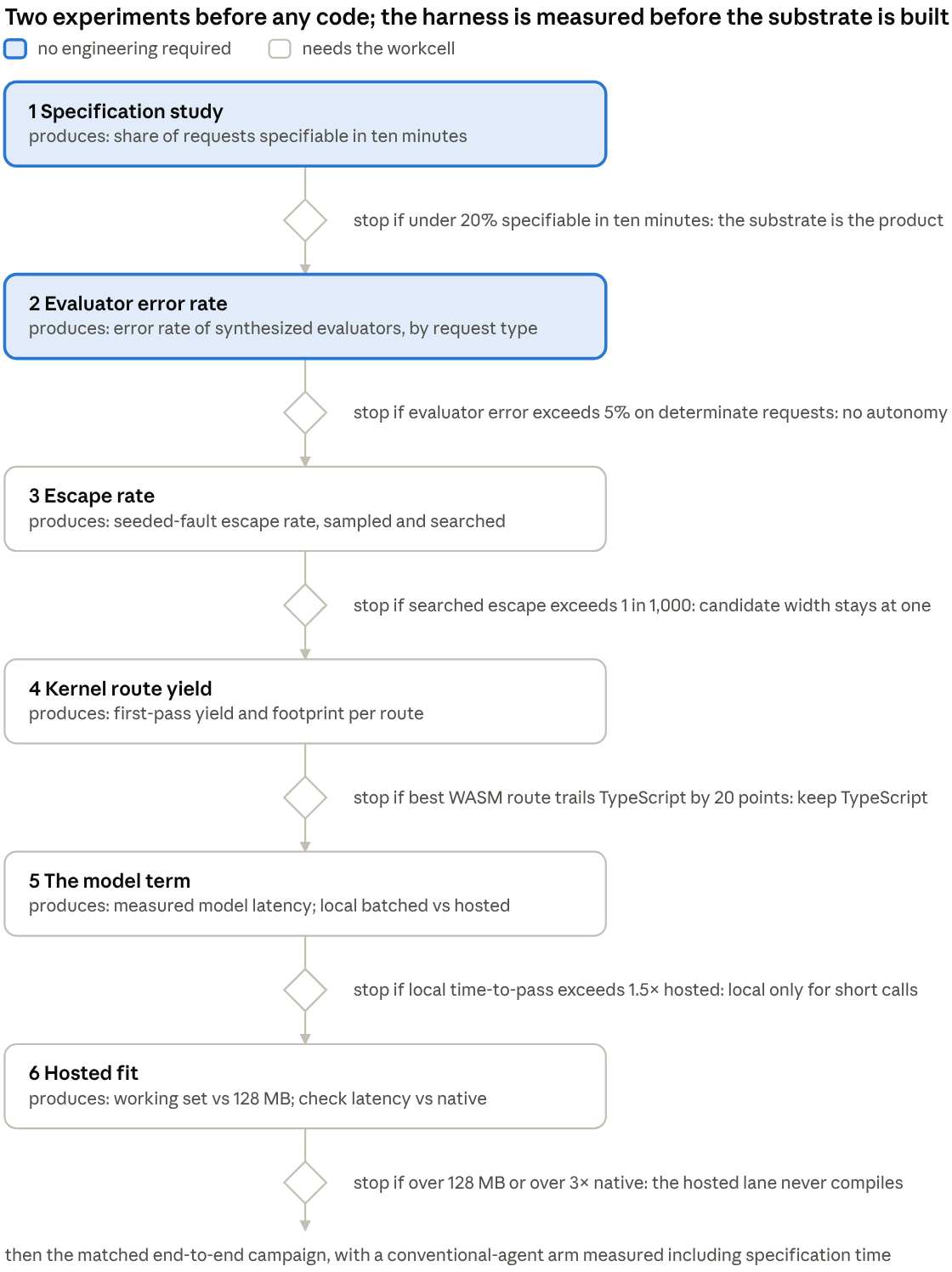
Observation	Value	Label and scope
Test scope	180 tests on the published checkout: 170 passes, 10 Linux skips; the broader dirty-worktree run remains failed at 2,106 tests with 17 failures and 325 errors	Recorded; two different scopes, not one green suite
Local model trial	27B GGUF, CPU-only, two threads: loaded in 4.14 s, no completed JSON within the 90 s deadline, 9.83 GiB peak	Recorded; evidence of bounded cleanup, not of local inference
tsc-rs native, T3 Code	7.95 s vs TypeScript 7 at 19.45 s (2.45×); fast mode 6.91 s at up to 2.2× CPU and 1.9× peak memory	Upstream, 13 Oct 2026, 16 vCPU runners; native, single project set, not WebAssembly
tsc-rs WebAssembly interface	4.2 MB module, 1.8 MB gzip; compile once, new instance per check; single-threaded; no watch or language-service mode	Upstream documentation; an embedding surface, not a completed integration

What these do not establish, stated once so no later section can lean on them: the compiler running inside the intended isolate; any model in the loop; a synthesized evaluator's error rate; an escape rate on seeded faults; a generated capability helping a real person with a real task; a search campaign beating a baseline at equal budget; density or energy on unified-memory hardware; multi-tenant security; physical power-loss survival.

The guest numbers above describe the adapter this edition replaces. They are kept because they are the baseline Experiment 4 must beat, and because the containment and recovery behavior they record is the part of the earlier design that carries forward unchanged.

15. The first six experiments

Six experiments, in this order, replace the next architecture document; each produces one number and each number decides something, and the first two need no engineering at all.



experiment sequence · 6 experiments, 6 gates

Each gate is a stop rule written before the result is known; a failed gate narrows the product rather than the standard.

	Experiment	What it produces	Decision it drives	Kill criterion
1	**Specification study.** Twenty real requests (Field OS users or your own work); hand-write the acceptance contract for each and time it	Fraction specifiable in under ten minutes; fraction satisfiable by composing what exists; size of the residual	Whether generative applications have a market now, and how thin a residual to design for	Under 20% specifiable in ten minutes: the agent substrate is the product; generative applications move to research
2	**Evaluator error rate.** One hundred tasks; POODO synthesizes the residual in a candidate-blind context; people grade the evaluators, not the candidates	Evaluator error rate by request type, before and after one pass of invariant-library growth	The starting autonomy tier for any domain; the real size of the trust gap	Above 5% on determinate requests after two library passes: no autonomous tier anywhere; every domain starts Confirmed
3	**Escape rate.** Seed known faults into candidates; run the full harness; count promotions, first under random sampling, then inside a sixteen-wide campaign	Escape rate per domain, sampled and searched	Tier assignment per domain; whether candidate width may rise	Searched escape rate above 1 in 1,000 with the full harness: width stays at one until a new non-model check lands
4	**Kernel route yield.** The same twenty tasks generated in TypeScript, AssemblyScript, and Rust; first-pass yield, tokens, compile time, per-candidate memory, escape rate	Yield per route; module footprint against the 150 MiB guest baseline	The default route; whether removing the interpreter layer is justified now	Best in-isolate WebAssembly route more than 20 points below TypeScript: the compatibility lane stays default and the kernel contract waits

	Experiment	What it produces	Decision it drives	Kill criterion
5	**The model term.** One model turn against the actual SDK context with prompt caching; then four-way batched generation of a 1,000-token kernel on the target unified-memory machine against one hosted stream, with compilation running concurrently	The first measured model latency in this project; time to first passing candidate, local versus hosted; bandwidth contention	Inference placement; whether unified memory is a lane or a preference	Local batched time-to-pass, yield-adjusted, worse than 1.5× hosted: local inference serves short calls only
6	**Hosted fit.** tsc-rs as WebAssembly plus harness plus one module instance inside a Workers isolate; working set against 128 MB; a 50-file overlay check against native; Dynamic Worker creation cost	Fits or does not; check latency ratio; creation cost per candidate	Whether the hosted lane compiles or only runs pre-checked artifacts	Over the memory limit, or more than 3× native: the hosted lane consumes checked artifacts and never compiles

After the six comes the matched end-to-end campaign of Edition 2′s design: identical tasks, frozen evaluators, and arms for a warm native baseline, the kernel workcell, and a child-isolate topology. This edition adds the arm the earlier plan lacked: a conventional agent in a warm container, measured end to end including the time to specify. The product proposition is only confirmed against that arm, not against a cold VM.

Each experiment is days of work, not months. Together they are smaller than the three documents they replace, and they answer the questions those documents could only propose.

16. Prior art and positioning

Almost every mechanism in this design existed before it; the new ingredient is that proposals are cheap, and the honest claim is a trust-and-economics architecture for cheap proposals, not a new paradigm of software creation.

Lineage	What it established	What Endoforge takes	Where it differs
Live programming environments (Smalltalk, Lisp images, Pharo)	The runtime can contain its own compiler and grow while running	The whole premise of software creation as a runtime operation	Proposals come from a model, so qualification and authority become the design problem
End-user programming (spreadsheets, notebooks; Nardi, Ko)	People want results, not software; they rarely verify; generated artifacts go unmaintained	The demand-side caution behind Section 3 and Experiment 1	A contract the person confirms, and an inventory discipline for what is kept
Search-based software engineering and genetic improvement (GenProg onward)	Mutate, evaluate on tests, keep improvements; patches overfit their test suite	Bounded frontiers, held-out confirmation, diversity	The evaluator is synthesized per task and qualified by mutation before it may promote
LLM-guided evolutionary search (FunSearch, AlphaEvolve)	Model proposals plus automated selection work where a cheap exact evaluator exists	The campaign structure of Section 11	Most requests here have no exact evaluator, which is why Section 6 exists
Content-addressed code (Unison, Nix, Bazel action caches)	Definitions named by hash; derived results keyed by all inputs; no builds	Artifact identity, build keys, derivation caching	A hash names bytes and never grants permission or proves task correctness

Lineage	What it established	What Endoforge takes	Where it differs
Incremental computation (Salsa and rust-analyzer, tsserver builder, Roslyn)	Change-proportional re-analysis is engineering, not research	The far-end goal of work proportional to change	Applied to candidate families and evaluations, not only to a single project's semantics
Durable execution (Temporal, DBOS, Durable Objects)	Durable tasks outlive attempts; outcome-unknown is a first-class state	Section 9's lifetimes and recovery rules	Attempts run untrusted generated code, so recovery must also reason about containment
Warm sandboxes and microVMs (Firecracker snapshot restore, E2B, Modal)	Environment setup can be amortized to roughly 100 ms	The baseline the agent substrate must beat	A module instance per candidate rather than a VM per environment; the evaluator is part of the product
WebAssembly runtimes and the Component Model (Wasmtime, Spin, WIT)	Per-request fresh instances from a pre-compiled module in under a millisecond; typed language-independent interfaces	The kernel contract and the local lane	The host runs an untrusted-proposal loop with provenance and promotion around those instances
Cloudflare Code Mode, Dynamic Workers, Project Think, workerd	Runtime-created software, generated programs composing tools, durable agents	The hosted lane and its limits	The complete qualified loop and its measured economics, which no platform primitive supplies

Lineage	What it established	What Endoforge takes	Where it differs
Object-capability security (E, Cap'n Proto) and the Workers security model	Capabilities as references; isolates are not a side-channel boundary without platform mitigations	The broker, declared imports, cohort partitioning	Candidates are generated per request, so the capability surface is renegotiated per admission

The positioning that follows from the table is deliberately modest. The agent substrate competes with warm sandboxes on density and locality, and with agent harnesses on the quality of its qualification loop. Generative applications compete with configurable software and spreadsheets, and win only where Experiment 1 shows a residual that composition cannot cover and specification cost does not swamp. Neither product is a platform replacement, and this document does not describe one.

17. The claim ladder

Each rung below is a different claim with a different kind of evidence, and the purpose of the design is to climb them in order without letting any rung stand in for the one above it.

Rung	Claim	Kind of result	Status, 17 October 2026
1	A compiler runs inside the workcell	Embedding	Not yet; tsc-rs is an embedding surface, and no in-isolate route has been exercised
2	A candidate module executes bounded, contained, and recoverable	Execution	Recorded for the previous guest adapter; not yet for the kernel contract
3	A synthesized evaluator has a measured error rate and a measured escape rate	Specification	Not yet; Experiments 2 and 3
4	A generated capability completes a real person's real task	Task	Not yet; no generative UI has been qualified with a user

Rung	Claim	Kind of result	Status, 17 October 2026
5	A campaign produces more accepted improvement at equal total cost than a baseline	Optimization	Not yet; no campaign has been run
6	Retained capabilities make later requests cheaper over months of use	Lifetime economics	Not yet; the library does not exist
7	The factory is more effective after accepting changes to its own machinery	Recursive engineering	Not yet; requires rungs 3 and 5

What this document does not claim: a universal speedup; that the underlying primitives are new; that agents or any harness rarely make mistakes; that the model can be trusted at run time; that a hash or a receipt is permission; that in-isolate locality is the main source of value; that the physical-limits discussion predicts anything about this system's performance; that any number labeled estimated has been measured; or that Field OS is a released multi-tenant generative platform.

What it does claim: that the design is internally consistent after the changes of this edition; that its trust model holds whether models improve quickly or slowly, because autonomy is a measured dial rather than an assumption; that the six experiments in Section 15 are sufficient to decide which of the two products is real and in what order to build it; and that the first two of those experiments can begin tomorrow.

18. Sources and identities

Every recorded figure traces to one of the receipts below; every upstream figure to a pinned public source; every estimate to this edition's own reasoning, which Section 15 is designed to replace with measurements.

Repository receipts (private, Akilleez-QA/field-ops at 58d6826b2ac66d7d7ca7edd2f2e1bd869b5abe18, 8 October 2026). R01 Worker Fabric v1 specification. R03 runtime guide for the private adapter. R04 main-rig execution receipt (cold and warm tables, oracle scope, failed model trial). R05 W1/W2 receipt (seven gateway cases, containment, broad-suite failures). R06 W3/W4 receipt (return and recovery tests, published-checkout scope). R07 admission and fence implementation. R08 supervised execution and host accounting. R09 guest bridge. R11 instrument binding

contract. Frozen identities: shared domain bundle 51,998 bytes, SHA-256 a82e3073…231e; six-case output SHA-256 1c5facb2…033d. None of these receipts was rerun for this edition.

Upstream, read for this edition. ts-rust releases: first public preview 7 October 2026; an LLM-written Rust port of the TypeScript 7 compiler; native benchmarks as documented in the repository README on 13 October 2026. Cloudflare Workers limits: 128 MB per isolate including WebAssembly allocations; 10 MB compressed Worker size on paid plans. POODO: the reasoning and orchestration skill whose precommit record and claim states this design adopts, and whose repository states that behavioral improvement is unmeasured.

Upstream, carried from Edition 2 and not reverified here. ts-rust WebAssembly interface (U02): 4.2 MB module, compile once and instantiate per check, single-threaded. Cloudflare Dynamic Workers getting-started, API reference, and platform limits (U04 to U06), checked 14 October 2026: four in-flight Dynamic Workers per request; ten per Durable Object I/O context. Workers performance and timers (U09). Cloudflare Code Mode (U11). workerd README and the pinned server source with its null isolate limit enforcers (U13, U14). Mononoke source study at 79071c2a3ea20c36ed71332608535e5801101bba, 16 October 2026: nineteen files inspected, no build or tests run.

Edition history. The In-Isolate Software Workshop, 14 October 2026, version 1.0, 25 pages. The Diagram Atlas, 14 October 2026, thirty sheets. Endoforge: The Entire Idea, Edition 2, 16 October 2026. The eighteen-document engineering reference, work packages, and campaign templates under 05_Engineering_Reference. This edition supersedes their architecture where it differs and preserves their evidence boundaries where it does not.

Revalidation. Platform limits, compiler releases, and runtime behavior change without notice. Every upstream figure above is to be rechecked against the exact build and host before the experiment that depends on it, and the as-of date in the byline is the date of this reading, not a warranty.